

KERNEL MODULE REPORT

Abstract:

This report focuses on developing a secure kernel-level system that monitors external peripheral and monitor connections during digital exams. The system includes several Linux kernel modules and a user-space utility designed to log and, if necessary, respond to suspicious hardware events. By detecting USB HID devices, HDMI/VGA connections, Bluetooth pairing, and Ethernet plug-ins at the kernel level, the system ensures tamper-resistant event tracking. In specific scenarios, such as multiple monitor connections, the system is configured to initiate an automatic shutdown to prevent cheating or information leaks.

Individual Module Implementations

1. HDMI and USB HID Detection: `exam_alert.c`

What it does: This module monitors USB input devices and HDMI monitor connections to identify any new device attachments during exam sessions. If two or more such devices are detected, it initiates a system shutdown to prevent unauthorized use.

This module intercepts USB HID device insertion (mouse, keyboard, others) and framebuffer notifications for HDMI monitor connections. It uses the `usb_register_notify` and `fb_register_client` kernel APIs. For each device event, the kernel logs an entry with the vendor ID, product ID, interface type, and current time converted to IST using `ktime_get_real_ts64()` and a manual offset of 19800 seconds. Upon detecting any two devices (USB + HDMI, or two USBs), a controlled system shutdown is triggered using `call_usermodehelper()`.

Key Headers and APIs:

1. `#include <linux/usb.h>`
2. `#include <linux/fb.h>`
3. `#include <linux/reboot.h>`
4. `#include <linux/timekeeping.h>`

Major Changes Made:

- Integrated notifier chains
- Created global event counters
- Added shutdown hook using `call_usermodehelper()`

C-code

[exam_alert.c](#)

2. Bluetooth ACL Connection Logger: `bt_kprobe.c`

What it does: This module logs every new Bluetooth ACL connection by probing the kernel's internal Bluetooth connection function. It helps identify any unexpected wireless communication attempts.

This kernel module uses kprobes to intercept the `hci_connect_acl` function, which is internally called during Bluetooth ACL link setup. The MAC address of the remote device is extracted from the `conn->dst` field and logged to the kernel using `printk()`. To avoid redundant logs, a static array of MACs (`known_addrs`) is maintained to filter duplicates.

Key Headers and APIs:

1. `#include <linux/kprobes.h>`
2. `#include <net/bluetooth/hci_core.h>`

Major Changes Made:

- Added pre-handler for `hci_connect_acl`
- Extracted and formatted MAC address
- Included deduplication logic for known device

C-code

[bt_kprobe.c](#)

3. Ethernet Interface Logger: `eth_alert.c`

What it does: This module tracks the activation of wired network interfaces. It logs Ethernet interface names and the timestamp, useful for identifying unauthorized physical network access.

This module registers a notifier using `register_netdevice_notifier` to listen for `NETDEV_UP` events. It filters interfaces of type `ARPHRD_ETHER` (Ethernet). Upon detection, it logs the interface name and the time (formatted to IST) using manual offset adjustment with `ktime_get_real_ts64()`.

Key Headers and APIs:

1. `#include <linux/netdevice.h>`
2. `#include <linux/timekeeping.h>`

Major Changes Made:

- Event filter for Ethernet interfaces only
- Timestamped logging using `printk()`
- Indexed log tracking via a counter

C-code

[eth_alert.c](#)

4. Monitor Detection Interface: `monitor_detector.c`

What it does: This module creates a communication bridge between user-space monitoring tools and the kernel. It receives and logs monitor connection status messages (connected/disconnected) for auditing.

This kernel module sets up a procfs entry `/proc/monitor_status` and accepts string commands from user space in the format `connected HDMI-A-1` or `disconnected VGA-1`. Each entry is parsed using `sscanf()` and logged with a time-stamped count. This allows separation of logic between the user-space poller and kernel event recording.

Key Headers and APIs:

1. `#include <linux/proc_fs.h>`
2. `#include <linux/uaccess.h>`
3. `#include <linux/timekeeping.h>`

Major Changes Made:

- Created proc entry with `proc_create()`
- Implemented `write` callback for parsing input
- Integrated timestamped kernel logging

5. User-space Monitor Status Poller: `user_monitor_watcher.c`

What it does: This user-space program scans connected monitors via DRM interface and informs the kernel module of any status changes, enabling the kernel to log them securely.

This C program runs in user space and polls DRM interface files under `/sys/class/drm/`. It compares current monitor status with previously cached states. If a change is found (e.g., monitor connected/disconnected), it writes the corresponding status string into `/proc/monitor_status`, which is then logged by the kernel module.

Key Functions Used:

`opendir()` `readdir()` `fopen()` `fgets()` `system()`

Major Changes Made:

- Implemented polling every 2 seconds
- Filtered only external connectors (skipped eDP)
- Constructed echo commands to communicate with procs

C-code

1. [Monitor_detector.c](#)
2. [user_monitor_watcher.c](#)

Testing & Results

Each module was tested in isolation and in combination. For HDMI and USB modules, physical connections were made and monitored using `dmesg`. Bluetooth devices were paired from smartphones, and the MAC addresses were captured. Ethernet testing involved cable hot-plugging. Monitor status changes were confirmed by checking `/sys/class/drm/*/status` and verifying corresponding logs in `dmesg`. The system shutdown was verified by connecting a combination of USB and HDMI devices.

Future Work

The system can be enhanced by developing heuristic-based splitter detection using EDID analysis or connector topology. A desktop interface could be built to provide live alerts, and sysfs toggles could be introduced to control shutdown behavior . Exporting logs to a centralized dashboard is another area of potential improvement.

Appendices

- Source code listings: `exam_alert.c`, `bt_kprobe.c`, `eth_alert.c`, `monitor_detector.c`, `user_monitor_watcher.c`
- Build scripts and Makefiles
- `dmesg` outputs for each detection scenario

